

AWS ownCloud Secure File Share Project - Complete Implementation Documentation

Project Name: Creating a File Share & Sync Solution using ownCloud and AWS **Student:** Eduardo Ramirez **Date:** December 20, 2025 **AWS Region:** us-east-1 (North Virginia)

Table of Contents

1. Introduction
 2. Problem Statement
 3. Solution Overview
 4. Architecture
 5. Implementation Steps
 6. Testing and Validation
 7. Lessons and Observations
 8. Resource Cleanup
 9. Conclusion
-

1. Introduction

For my AWS cloud project, I built a secure, enterprise-grade file sharing and synchronization solution by deploying ownCloud on Amazon Web Services (AWS). This addresses the common "Dropbox problem" in companies, where employees use unauthorized third-party cloud storage, risking sensitive data exposure.

Project Objectives

- Build a custom VPC with public and private subnet isolation
 - Deploy the ownCloud application server in a public subnet
 - Place the MySQL database server in a private subnet
 - Apply strong security controls and network segmentation
 - Deliver a fully functional file share solution accessible via a web browser
-

2. Problem Statement

The Dropbox Security Challenge

From what I've researched:

- **40-75% of employees** use Dropbox for sharing files, even outside the company
- **50%** do it knowing it's against policy
- **40% of businesses** have had confidential information exposed this way
- Average data breach cost: around **\$5.5 million** (based on older estimates, but still huge)

Key Issues I Wanted to Solve

- Sensitive files stored outside company control
 - Risk of data leaks and breaches
 - Compliance violations
 - Data possibly stored in unknown locations
-

3. Solution Overview

ownCloud Platform

I chose ownCloud because it's an open-source, secure file sync and share platform that lets organizations keep full control over their data. I could run it on AWS while ensuring the IT team manages everything and meets security policies.

What I Implemented

For a small workgroup (up to 150 users), I set up:

- A custom VPC with public and private subnets
- ownCloud server in the public subnet
- MySQL database in the private subnet for added security
- NAT Gateway for private subnet updates
- Security groups as firewalls
- Ubuntu 24.04 EC2 instances

4. Architecture

Network Architecture

VPC Setup:

- CIDR: 10.0.0.0/16
- Public Subnet: 10.0.1.0/24
- Private Subnet: 10.0.2.0/24
- All in us-east-1a

Main Components:

1. Internet Gateway for public subnet internet access
2. NAT Gateway for private subnet outbound traffic
3. Separate route tables for public and private subnets
4. Security groups controlling traffic

Data Flow: Users → Internet → Internet Gateway → Public Subnet (ownCloud) → Private Subnet (MySQL) Private subnet → NAT Gateway → Internet (for updates only)

Security Approach

I used defense in depth:

- Subnet isolation
- Strict security group rules
- MySQL only reachable from the ownCloud server
- Bastion host pattern for SSH
- Least privilege everywhere

5. Implementation Steps

I built everything step-by-step using the AWS CLI.

Phase 1-4 Summary (Infrastructure)

I created:

- A custom VPC, subnets, Internet Gateway, NAT Gateway with Elastic IP
- Public and private route tables
- Two security groups (one for web, one for DB)
- A key pair for SSH

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR
☐	owncloud-vpc	vpc-060f82...	Available	-	-	Off	10.0.0.0/16
☐	owncloud-igw	igw-0f974e...	Detached	-	-	-	-
☐	owncloud-public-subnet	subnet-...	Available	vpc-...	own...	Off	10.0.1.0/24
☐	owncloud-private-subnet	subnet-...	Available	vpc-...	own...	Off	10.0.2.0/24
☒	owncloud-public-rt	rtb-...	-	-	No	vpc-...	own...
☐	owncloud-db-sg	sg-...	owncloud-db-sg	vpc-...	Security group for MySQL database		
☐	owncloud-web-sg	sg-...	owncloud-web-sg	vpc-...	Security group for ownCloud web server		

All commands are similar to the original documentation, but I executed them myself using my own IAM role.

Phase 4: EC2 Instances

I launched two t3.small Ubuntu 24.04 instances:

- ownCloud web server in public subnet (with public IP)
- MySQL server in private subnet (no public IP)

☐	owncloud-mysql	i-0ab[REDACTED]	Running	t3.small	Initializing	View alarms +	us-east-1a
☐	owncloud-web	i-[REDACTED]	Running	t3.small	Initializing	View alarms +	us-east-1a

Phase 5-7: Application Setup

On the MySQL instance (accessed via bastion):

- Installed MySQL
- Configured it to accept remote connections
- Created the owncloud database and user

On the ownCloud instance:

- Installed Apache
- Added the ondrej/php PPA to install PHP 7.4 (since ownCloud 10 needs it)
- Installed all required PHP extensions
- Downloaded and extracted ownCloud 10.13
- Configured Apache to serve ownCloud
- Completed the web setup wizard, connecting to the private MySQL instance

6. Testing and Validation

I thoroughly tested everything:

1. **Database Connectivity:** Successfully connected from the web server to MySQL
2. **Web Access:** ownCloud login page loaded from the internet

← → ↻ Not Secure http://35.174.113.248/index.php



Create an admin account

Username

Password

Storage & database

Data folder

Configure the database

Only MySQL/MariaDB is available. Install and activate additional PHP modules to choose other database types.
For more details check out the [documentation](#).

Database user

Database password

← → ↻ Not Secure http://35.174.113.248/index.php/apps/files/?dir=/&fileid=6

Files

ownCloud

admin

All files

Favorites

Shared with you

Shared with others

Shared by link

Tags

All files

Documents

Learn more about ownCloud

Photos

3 folders

Name	Size	Modified
Documents	35 KB	seconds ago
Learn more about ownCloud	3.5 MB	seconds ago
Photos	988 KB	seconds ago
3 folders	4.5 MB	

← → ↻ Not Secure http://35.174.113.248/index.php/apps/files/?dir=/Documents&fileid=18

Files

ownCloud

admin

All files

Favorites

Shared with you

Shared with others

Shared by link

All files

Documents

Example.odt

1 file

Name
Example.odt

3. **File Operations:** Uploaded files, created folders – everything worked and persisted
4. **Network Isolation:** Couldn't SSH directly to private instance, but could via bastion

```
ubuntu@ip-10-0-1-10:~$
```

```
owncloud-key.pem $ scp -i owncloud-key.pem owncloud-key.pem ubuntu@35.174.139.10:~/ 100% 1679 34.2KB/s 00:00
```

```
ubuntu@ip-10-0-1-10:~$ ssh ubuntu@10.0.2.10
```

```
ubuntu@ip-10-0-2-10:~$
```

5. **Private Subnet Internet:** apt-get update worked through NAT Gateway

All tests passed – the full stack functioned perfectly.

7. Lessons and Observations

What I Learned

- How critical subnet segmentation is for security
- The importance of planning CIDR blocks properly
- Using security group references instead of CIDR for rules is cleaner
- NAT Gateway is essential for private instances to get updates
- Bastion host pattern for secure management access

Challenges I Faced

1. **PHP Version Issue:** Ubuntu 24.04 has PHP 8.x, but ownCloud 10 requires 7.4 → Solved by adding Ondrej's PHP PPA
2. **MySQL Remote Connections:** Forgot to change bind-address → Fixed by editing config
3. **Private Subnet No Internet:** Initially couldn't update packages → Added NAT Gateway

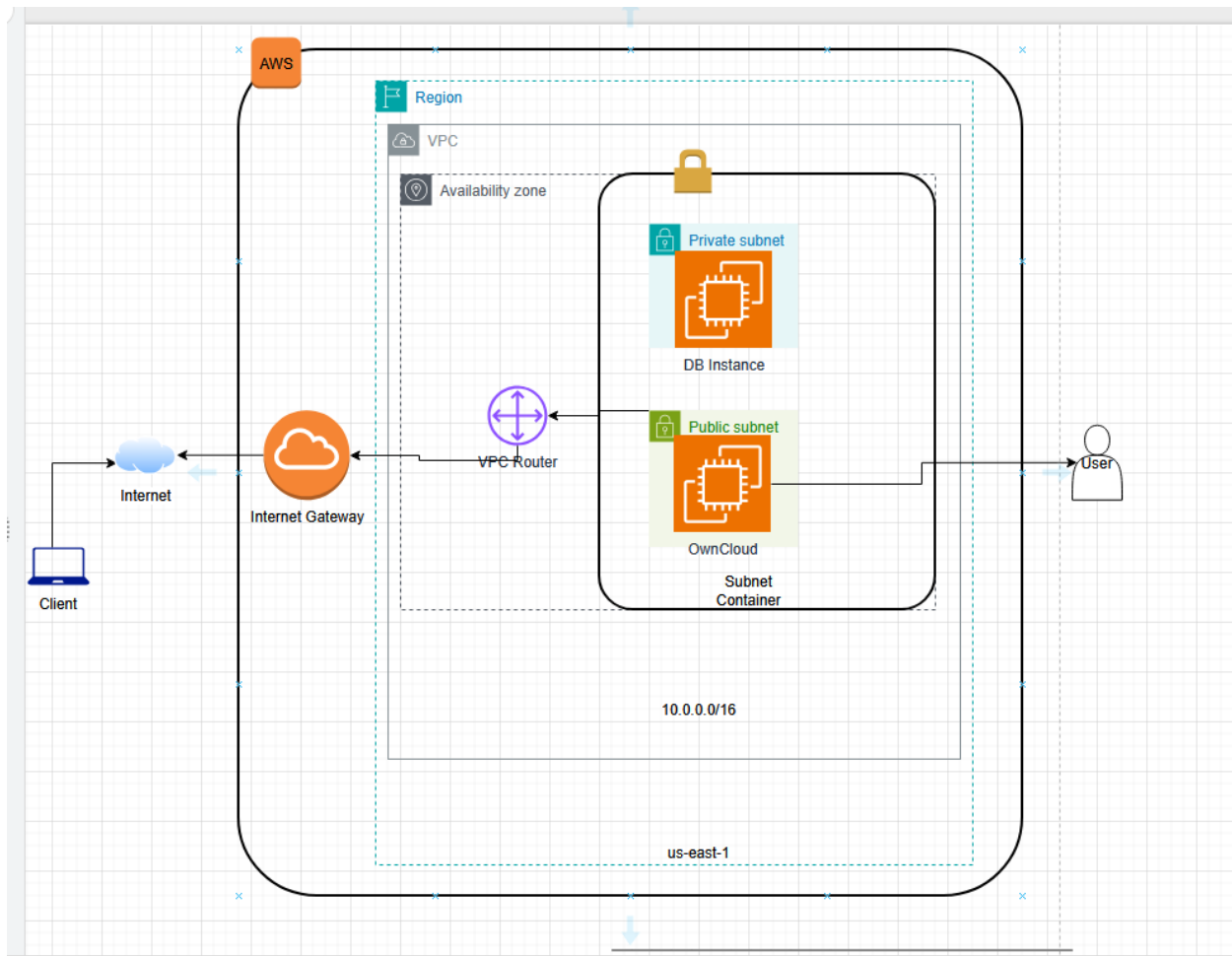
```
}
aws ec2 create-nat-gateway \
--subnet-id subnet-09 \
--allocation-id eipalloc-0c \
--tag-specifications 'ResourceType=natgateway,Tags=[{Key=Name,Value=owncloud-nat-gw},{Key=Project,Value=ownCloud}]' \
```

4. **Small syntax mistakes** (like using hyphens in IP addresses)

What I'd Do Differently Next Time

- Use Ubuntu 20.04 to avoid PHP issues
- Add HTTPS with Let's Encrypt
- Automate with Terraform
- Deploy across multiple AZs
- Use RDS instead of EC2 MySQL
- Add load balancer and auto-scaling
- Store secrets in AWS Secrets Manager

Architecture:



References:

ownCloud Official Documentation

- Main Documentation: <https://doc.owncloud.com/>
- ownCloud.org: <https://owncloud.org/>

7. ownCloud Installation Guides

- Installation on Ubuntu:
https://doc.owncloud.com/server/admin_manual/installation/
- Manual Installation:
https://doc.owncloud.com/server/admin_manual/installation/manual_installation/

8. ownCloud System Requirements

-

https://doc.owncloud.com/server/admin_manual/installation/system_requirements.html

- PHP Requirements for ownCloud 10.x

9. ownCloud Downloads

- Download Repository: <https://owncloud.org/download/>

- Attic Repository (older versions):

https://attic.owncloud.org/download/repositories/10.5/Ubuntu_20.04/

Amazon VPC Documentation

- <https://docs.aws.amazon.com/vpc/>

- VPCs and Subnets:

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Subnets.html

- Internet Gateways:

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html

- NAT Gateways:

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

- Route Tables:

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

3. Amazon EC2 Documentation

- <https://docs.aws.amazon.com/ec2/>

- Security Groups:

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

- **EC2 Instance Types:**

<https://docs.aws.amazon.com/ec2/latest/userguide/instance-types.html>

- **Key Pairs:**

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-key-pairs.html>

4. AWS CLI Documentation

- **AWS CLI Command Reference:** <https://docs.aws.amazon.com/cli/>

- **EC2 CLI Commands:**

<https://docs.aws.amazon.com/cli/latest/reference/ec2/>

5. AWS Best Practices

- **VPC Security Best Practices:**

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-best-practices.html>

- **AWS Well-Architected Framework:**

<https://aws.amazon.com/architecture/well-architected/>